

Relatório de Análise dos Exemplos da Aula 07

Expressões, atribuição e estruturas de controle em diferentes linguagens

Gabriel Galhardone Felipe
RA: 24021292-2

Introdução

Este relatório reúne a análise dos 11 grupos de exemplos contidos no arquivo aula007.rar. Ao todo são 49 programas, escritos em C, Java, Python, JavaScript, Go, Rust, PHP, Haskell, Ruby e Lisp. Para cada grupo são apresentados o objetivo, a explicação do que cada arquivo faz, os resultados que se espera obter e uma conclusão curta.

O fio condutor do material é simples: um trecho de lógica aparentemente igual pode produzir resultados diferentes conforme a linguagem. As regras de precedência dos operadores, a conversão entre tipos, a ordem de avaliação, a atribuição e os comandos de seleção e repetição mudam de um ambiente para outro, e entender essas diferenças é o que evita erros difíceis de perceber.

A análise se baseia apenas nos códigos e nos comentários fornecidos. Por isso, as saídas descritas são resultados esperados a partir da leitura dos programas, e não o registro de uma execução de todos eles. As linhas comentadas que provocam erro existem como demonstração e não participam da execução normal. Quando um resultado é aleatório ou depende do compilador, isso é dito explicitamente no texto.

Visão geral dos grupos

A tabela a seguir resume o tema de cada grupo, as linguagens envolvidas e a quantidade de programas analisados.

Nº	Tema	Linguagens	Arquivos
1	Precedência e associatividade	Python, JavaScript, Java	3
2	Efeitos colaterais e ordem de avaliação	Java, C, Go	3
3	Sobrecarga de operadores	Python, Rust, Java	3
4	Conversões de tipos	Java, Go, Rust, JavaScript, PHP	5
5	Expressões booleanas e curto circuito	C, Python, Java, JavaScript	5
6	Atribuição e modo misto	C, Python, Go, Rust, Java	6
7	Estruturas de seleção	C, Go, Java, Python, Rust	6
8	Estruturas de iteração	C, Python, JavaScript, Java, Rust	6
9	Desvios e comandos protegidos	C, Python, Go, Haskell	4
10	Exercícios de identificação de erros	JavaScript, Python, Java, C, Go	6
11	Autopesquisa sobre Ruby e Lisp	Ruby, Lisp	2

1 Precedência e associatividade

Objetivo

Demonstrar como a prioridade entre os operadores, a maneira de agrupar subexpressões e o tipo dos operandos determinam o valor final de uma expressão. Os três arquivos partem de contas simples, escolhidas justamente para expor armadilhas comuns de leitura.

Análise dos arquivos

precedencia.py – A multiplicação é resolvida antes da soma, de modo que $a + b * c$ resulta em 23, enquanto $(a + b) * c$ resulta em 35 porque os parênteses impõem outro agrupamento. Na potência, o operador tem precedência maior que o menos unário: $-2 ** 2$ vale -4, e somente $(-2) ** 2$ vale 4. A potência também associa da direita para a esquerda, então $2 ** 3 ** 2$ equivale a 2 elevado a 9, isto é, 512; se o agrupamento fosse feito pela esquerda, o valor seria 64. A subtração, por sua vez, associa pela esquerda, e $10 - 4 - 3$ vale 3. Com números negativos, a divisão inteira arredonda para baixo ($-7 // 2$ resulta em -4) e o resto acompanha o sinal do divisor ($-7 \% 2$ resulta em 1).

precedencia.js – As expressões de soma e de potência dão os mesmos 23 e 512. A diferença aparece na sintaxe: o JavaScript não aceita o menos unário aplicado diretamente à base de `**`, o que obriga o programador a escolher entre $(-2) ** 2$, que vale 4, e $-(2 ** 2)$, que vale -4. O arquivo também mostra o efeito do operador `+` com texto: $1 + 2 + "3"$ gera "33", pois a soma numérica acontece primeiro, e $"1" + 2 + 3$ gera "123", já que, a partir do primeiro texto, tudo passa a ser concatenação.

Precedencia.java – O operador `+` associa à esquerda, por isso $1 + 2 + "3" + 4 + 5$ resulta em "3345": primeiro $1 + 2$ vira 3 e os demais valores são anexados como texto. Com texto no início, $"x" + 1 + 2$ gera "x12", mas $"x" + (1 + 2)$ gera "x3". Em $7 / 2 * 2.0$, a divisão inteira $7 / 2$ ocorre primeiro e vale 3, resultando em 6.0; ao inverter a ordem, $2.0 * 7 / 2$ produz 7.0. Para -7 e 2, a divisão e o resto valem -3 e -1, valores diferentes dos obtidos em Python, pois o Java trunca em direção a zero. Por fim, `Math.pow(2, 10)` devolve 1024.0, um número de ponto flutuante.

Comparação entre as linguagens

Um mesmo conjunto de números pode levar a respostas diferentes: a divisão de -7 por 2 dá -4 em Python e -3 em Java, e o resto dá 1 em Python e -1 em Java. Já o comportamento do `+` com textos aparece tanto em JavaScript quanto em Java, e em ambos a posição do primeiro texto na expressão decide onde termina a soma e onde começa a concatenação.

Conclusão

Os parênteses tornam o agrupamento explícito e eliminam dúvidas, mas não bastam sozinhos: também é preciso observar o tipo de cada operando. Uma mesma expressão pode ter resultados distintos em linguagens diferentes.

2 Efeitos colaterais e ordem de avaliação

Objetivo

Estudar expressões em que uma função altera uma variável que também é usada no mesmo cálculo, situação em que a ordem de avaliação passa a influenciar o resultado.

Análise dos arquivos

Ordem.java – A variável global `a` começa valendo 5. A função `fun1` altera `a` para 17 e retorna 3. Como o Java avalia os operandos da esquerda para a direita, em `a = a + fun1()` o valor 5 já foi lido antes da chamada, e o resultado final é `a = 8`; a alteração para 17 se perde. Na expressão `i++ + i++`, os valores usados na soma são 1 e 2, e ao final `j` vale 3 e `i` também vale 3.

ordem.c – A função também altera `a` para 17. O material registra a saída 20 com o GCC 14, mas esse número não deve ser tratado como resultado portátil: ao contrário do Java, o C não garante em que ordem os operandos são avaliados. O trecho comentado `i++ + i++` tem comportamento indefinido, porque modifica a mesma variável duas vezes sem sequenciamento adequado entre as modificações.

ordem.go – A linguagem estabelece uma ordem para as chamadas de função, mas isso não assegura que a leitura da variável `a` aconteça antes de `fun1` nesse cálculo. O 20 indicado no arquivo corresponde ao compilador usado na demonstração e não deve ser assumido para qualquer implementação.

Linguagem	Resultado registrado	A ordem é garantida?
Java	<code>a = 8</code>	Sim, operandos da esquerda para a direita
C	20 (GCC 14)	Não; <code>i++ + i++</code> é comportamento indefinido
Go	20 (compilador usado)	Só para as chamadas, não para a leitura da variável

Conclusão

Separar a chamada da função e a leitura da variável em instruções distintas evita depender de regras de avaliação sutis, que mudam entre linguagens e até entre compiladores.

3 Sobrecarga de operadores

Objetivo

Comparar os diferentes significados que um mesmo operador pode assumir conforme os tipos envolvidos e verificar se a linguagem permite ao programador definir operações para tipos criados por ele.

Análise dos arquivos

vetor.py – A classe `Vetor` implementa `__add__`, `__mul__`, `__eq__` e `__repr__`. A soma de `Vetor(1, 2)` com `Vetor(3, 4)` gera `Vetor(4, 6)`, a multiplicação por 3 gera `Vetor(12, 18)` e a

comparação com `Vetor(4, 6)` retorna `True`. Os exemplos finais produzem `[1, 2, 3]` e `"ababab"`, ilustrando que o próprio Python já usa `+` para concatenar listas e `*` para repetir textos.

ponto.rs – O trait `Add` define a soma de dois pontos coordenada por coordenada, e o resultado é `Ponto { x: 4, y: 6 }`. Com `PartialEq`, a comparação entre pontos retorna `true`. A multiplicação `p * 2`, deixada em comentário, não compila, pois a operação não foi implementada para esse tipo.

Sobrecarga.java – O operador `+` faz soma numérica ou concatenação, dependendo dos operandos, e as saídas são `3`, `"12"`, `98` e `"a1"`. O caractere `'a'` entra na soma pelo seu valor numérico, `97`, o que explica o `98`. O Java não permite que o programador defina novos operadores para suas classes; por isso o `BigDecimal` recorre aos métodos `add` e `multiply` para chegar ao valor `0.9`.

Comparação entre as linguagens

Python e Rust permitem que a classe ou o tipo declare o que cada operador significa, cada um à sua maneira: métodos especiais no primeiro e traits no segundo. Em Rust, só o que foi implementado pode ser usado, e é por isso que `p * 2` falha na compilação. O Java adota o caminho oposto, com um conjunto fixo de significados para o `+`, e delega o restante a chamadas de método.

Conclusão

A sobrecarga torna o código mais natural quando se trabalha com tipos próprios, desde que o significado atribuído a cada operador seja claro e coerente com o que o leitor espera.

4 Conversões de tipos

Objetivo

Identificar onde ocorrem coerções automáticas e conversões explícitas, e quais delas resultam em perda de informação.

Análise dos arquivos

Conversoes.java – Converter `123456789` para `float` faz perder precisão: a saída é `1.2345679E8` e, ao converter de volta, o valor passa a ser `123456792`. O cast de `3.99` para `int` gera `3`, e o cast de `200` para `byte` gera `-56`. A soma de dois `byte` é promovida a `int` e exige um cast para voltar a `byte`, produzindo `30`. As divisões `1 / 2` e `1 / 2.0` resultam em `0` e `0.5`, conforme os operandos sejam ambos inteiros ou não.

conversoes.go – A conversão entre variáveis `int32` e `int64` precisa ser explícita. `int64(i)` produz `100` e `int(3.99)` produz `3`. Uma constante sem tipo, como `k`, pode ser usada em `k * 2.5`, com resultado `25`. Já a conversão de `300` para `byte` gera `44`, pois permanecem apenas os oito bits inferiores do número.

conversoes.rs – As conversões com as produzem `44` para `300` como `u8`, `4294967295` para `-1` como `u32` e `3` para `3.99` como `i32`. Ao converter `1e20` para `i32`, o valor satura em `2147483647`. Com `TryFrom`, o intervalo é verificado: `300` falha, enquanto `200` resulta em `Ok(200)`.

coercao.js – `"5" - 2` resulta em `3`, `"5" + 2` gera `"52"`, `"5" * "2"` gera `10` e `true + 1` gera `2`. Arrays e objetos viram texto quando somados: `[] + []` produz um texto vazio e `[] + {}` produz

"[object Object]". Além disso, `null + 1` vale 1 e `undefined + 1` gera NaN. A expressão que combina os textos b e a gera "baNaN". Nas comparações com `==`, os três exemplos produzem `true`, `true` e `false`; com `===`, os dois testes mostrados são `false`. Por fim, `Number("12px")` gera NaN, enquanto `parseInt("12px")` retorna 12.

coercao.php – As comparações `"0e123" == "0e456"`, `"1" == "01"` e `100 == "1e2"` são verdadeiras por conversão numérica, mas a comparação estrita entre as duas primeiras strings é falsa. No PHP 8, `"abc" == 0` é falso. Os dois hashes MD5 do exemplo são textos diferentes, porém o os interpreta como zero em notação científica e retorna `true`.

Linguagem	Conversão	Resultado
Java	3.99 para int	3
Java	200 para byte	-56
Java	123456789 para float e de volta	123456792
Go	300 para byte	44
Rust	300 como u8	44
Rust	-1 como u32	4294967295
Rust	1e20 como i32	2147483647 (saturação)

Conclusão

As conversões podem perder precisão ou mudar o sentido de uma comparação. No exemplo do PHP, usar a igualdade estrita evita a coerção que faz hashes diferentes parecerem iguais.

5 Expressões booleanas e curto circuito

Objetivo

Entender como funcionam as comparações, quais valores são tratados como verdadeiros ou falsos em cada linguagem e de que modo as condições são avaliadas apenas em parte.

Análise dos arquivos

relacionais.c – Com `a = 3`, `b = 2` e `c = 1`, a expressão `a > b > c` é lida como `(a > b) > c`, e o resultado é 0, pois 1 não é maior que 1. A forma `(a > b) && (b > c)` gera 1. No `if (x = 0)`, ocorre uma atribuição, e não uma comparação: o valor zero faz o programa executar o `else` e informar que x foi sobrescrito. Em C, o valor -1 é considerado verdadeiro, porque basta ser diferente de zero.

relacionais.py – O Python interpreta `a > b > c` como comparação encadeada e retorna `True`. São falsos o zero inteiro, o zero real, o texto vazio, a lista vazia, o dicionário vazio e o `None`. Em contrapartida, `"0"`, `[]` e um texto que contém apenas um espaço são verdadeiros.

CurtoCircuito.java – A busca termina com `indice = 3` sem acessar posição inválida, pois o `&&` verifica o limite do vetor antes. `false && caro()` não chama a função, mas `false & caro()`

chama, e por isso chamadas fica com 1. A verificação de uma string nula usando `&&` retorna false; com `&`, ocorre uma `NullPointerException`, que o exemplo captura.

curto_circuito.py – `False and caro()` e `True or caro()` não chamam `caro`. Os operadores devolvem um dos operandos: `0 or "padrão"` gera "padrão", texto vazio `or "anônimo"` gera "anônimo" e `[] and "nunca"` gera `[]`. Uma condição sobre `nome = None` retorna False sem avaliar `len(nome)`.

curto_circuito.js – O acesso `endereco?.cidade` retorna undefined sem lançar erro. Com `quantidade = 0`, a expressão `|| 10` devolve 10, mas `?? 10` preserva o zero. A expressão `usuario.nome && log.push(...)` acrescenta "tem nome" à lista.

Conclusão

O curto circuito evita chamadas desnecessárias e acessos inválidos. Em compensação, é preciso distinguir a ausência de valor de valores válidos como o zero, como mostra a diferença entre `||` e `??`.

6 Atribuição e modo misto

Objetivo

Comparar a atribuição simples, composta e múltipla, observar a questão da mutabilidade e analisar o que acontece quando se atribui um valor de um tipo a uma variável de outro.

Análise dos arquivos

atribuicao.c – A instrução `sum = count = 0` inicializa as duas variáveis com zero. Em `a = b + (c = d / b) - 1`, a variável `c` recebe 3 e `a` recebe 12. O laço `while` atribui e verifica cada caractere na mesma condição, imprimindo `[a][b][c]`. Depois de `+= 5` e de um incremento, `count` termina em 6.

atribuicao.py – A troca `a, b = b, a` resulta em 2 e 1. A desestruturação separa o 10 da lista `[20, 30]`. O operador `:=` guarda o tamanho da lista enquanto testa a condição, produzindo "lista longa: 6 elementos". Depois de `+= 5` e `//= 2`, `total` vale 7.

atribuicao.go – A declaração curta e a atribuição múltipla permitem trocar `a` e `b`, gerando 2 e 1. O `x++` é uma instrução, e não uma expressão, e por isso não pode compor uma atribuição a `y`. Após o incremento, `x` vale 11, e a inicialização dentro do `if` calcula `y = 22` e o imprime.

atribuicao.rs – As variáveis declaradas com `let` são imutáveis por padrão. Um novo `let x` cria outra variável com o mesmo nome e valor 6, o que se chama sombreamento. Com `let mut`, `total` pode ser alterado e chega a 10. A desestruturação gera 1 e 2. A atribuição produz o valor unitário `()`, que não serve como condição booleana.

modo_misto.c – O valor 3.7 é convertido para `int` e resulta em 3. O arquivo prevê 44 para `char c = 300` e 4294967295 para `unsigned int u = -1` no ambiente demonstrado. Esses números dependem da implementação: o tipo `char` e a largura de `unsigned int` podem variar, e a conversão de um valor fora do intervalo para `char` não deve ser considerada universal.

ModoMisto.java – As atribuições exibem 3.0, 3, 3 e A. A conversão de double para int exige cast, enquanto o alargamento de int para long é automático. A instrução `i += 3.7` embute uma conversão para o tipo de `i` e termina em 6, descartando a parte fracionária.

Conclusão

Cada linguagem tem suas próprias regras de atribuição. Operadores compostos, conversões implícitas e sombreado de variáveis merecem atenção para que o valor de uma variável não mude de forma inesperada.

7 Estruturas de seleção

Objetivo

Examinar os comandos `if`, `else`, `switch` e `match`, em especial a associação do `else` a um `if` e a passagem de um caso para o seguinte.

Análise dos arquivos

else_pendente.c – Sem chaves, o `else` pertence ao `if` mais interno, apesar do que sugere a indentação, e `result` termina em 1. Quando o `if` externo recebe chaves, o `else` passa a pertencer a ele: como `sum` é zero e `count` não é zero, `result` permanece em -1.

switch.c – Em `sem_break(1)`, os contadores `odd` e `even` são incrementados e ainda é impressa a mensagem de erro do `default`, porque a execução continua pelos casos seguintes. Em `com_break(1)`, o resultado é `odd = 1` e `even = 0`, sem a mensagem de erro.

switch.go – Cada caso termina sozinho. Para 1 e 3, é impresso "ímpar pequeno". Para 2, imprime-se "dois" e, por causa do `fallthrough` explícito, também "par pequeno"; para 4, "par pequeno"; para 9, "outro". O `switch` sem expressão classifica 42 como "menor que 100".

Switch.java – O `switch` com setas devolve uma classificação para cada dia: SEG a SEX são dias úteis e SAB e DOM são fim de semana. Como todos os valores do `enum` foram cobertos, não é preciso um `default`. Para `nota = 7`, a conta `nota / 2` vale 3 e o conceito é B. A palavra `yield` fornece o valor de um bloco dentro do `switch`.

selecao.py – O `match` estrutural retorna "encerrando", "indo para o norte", "comando desconhecido" e "pegando 2 item(ns)" para os quatro comandos testados. Uma guarda restringe as direções aceitas. A expressão condicional com `x = -3` gera `y = -6`.

selecao.rs – O `match` classifica -5 como negativo, 0 como zero, 7 como um dígito e 42 como vários dígitos; a cobertura dos casos precisa ser exaustiva. O `if` também pode produzir um valor: com `x = 10`, `y` recebe 10. No exemplo comentado, que pretende gerar um inteiro, a falta do `else` causa incompatibilidade de tipos.

Conclusão

Usar chaves e casos bem delimitados deixa o fluxo de seleção mais claro. As regras para a passagem entre casos e para o retorno de valores variam bastante entre as linguagens: o C continua pelos casos seguintes por padrão, ao passo que o Go encerra cada caso automaticamente.

8 Estruturas de iteração

Objetivo

Comparar laços de repetição, formas de percorrer coleções, saídas com rótulo e condições de término.

Análise dos arquivos

laco_for.c – O incremento de `i` dentro do corpo se soma ao incremento do próprio `for`, e a sequência impressa é 0, 3, 6 e 9. Um `for` com a atualização no corpo soma os números de 1 a 100 e gera 5050. O último laço usa duas variáveis e imprime (0,10), (1,9), (2,8), (3,7) e (4,6).

laco_for.py – Modificar `i` dentro do corpo não altera o próximo valor entregue por `range`, e por isso a primeira sequência segue de 0 a 9. `range(10, 0, -3)` produz 10, 7, 4 e 1. A função `enumerate` associa 0 a `ana` e 1 a `bia`. Depois do segundo laço, `i` continua disponível e vale 1.

iteracao.js – O `for...in` percorre as chaves "0", "1" e "2", e a soma vira concatenação, gerando "0012". Já o `for...of` percorre os valores e produz 60. O `forEach` mostra cada índice com o seu valor. O gerador `contagem` retorna a sequência [3, 2, 1].

Rotulos.java – O programa encontra -5 na posição (1,1), considerando que os índices começam em zero. O `break` com rótulo encerra os dois laços de uma vez. O `do while` imprime `k = 0` uma vez, mesmo com a condição final falsa, pois o teste ocorre depois do corpo.

rotulos.rs – O `break` rotulado também encerra os dois laços na posição (1,1). O `loop` pode produzir um valor com `break n`: duplicando `n` a partir de 1, a primeira potência de dois acima de 100 é 128.

zune.c – Com `dias = 366` e `ano = 2008`, a condição `dias > 365` é verdadeira, mas `dias > 366` é falsa. Nenhuma variável que controla a repetição se altera, e o algoritmo fica preso. O contador acrescentado à demonstração encerra o programa após 1.000.000 de voltas, com código de saída 1. Uma correção precisa permitir a saída quando os dias ainda pertencem ao ano atual, inclusive no limite do ano bissexto. Essa falha lembra o travamento que afetou o player Zune em 31 de dezembro de 2008.

Conclusão

Todo laço precisa de uma forma de avançar ou de terminar. Também é importante distinguir índices de valores e saber quem controla o próximo elemento da repetição: o corpo do laço ou o iterador.

9 Desvios e comandos protegidos

Objetivo

Analisar saltos na execução, guardas e a escolha entre alternativas habilitadas.

Análise dos arquivos

goto_fail.c – O segundo `goto fail` está fora do `if` e, por isso, executa sempre. A verificação da assinatura nunca é chamada, `err` permanece em zero e o programa informa "resultado = 0", o que se interpreta como certificado válido. No código demonstrado, remover o salto duplicado faz a

assinatura falsa produzir -1. O exemplo reproduz o tipo de erro que ficou conhecido no caso da biblioteca SSL da Apple, em 2014.

guardas.py – A lista inicial [9, 4, 7, 1] é ordenada por trocas entre vizinhos fora de ordem. A cada etapa, uma guarda verdadeira é escolhida de modo aleatório. A sequência de trocas varia de uma execução para outra, mas a saída final é sempre [1, 4, 7, 9]. Cada troca reduz o número de inversões, o que garante que o laço termine.

select.go – Os dois canais ficam prontos para leitura a cada rodada. O select escolhe uma das alternativas, contabiliza a escolha e esvazia o outro canal. Após 1.000 rodadas, as contagens de a e b somam 1.000; os valores individuais variam e não precisam ser exatamente 500 para cada lado.

guardas.hs – As guardas são testadas em ordem. maior 3 7 retorna 7, e a classificação de -2, 0 e 5 retorna negativo, zero e positivo. Quando mais de uma guarda é verdadeira, vale a primeira, diferentemente da escolha aleatória simulada em Python.

Arquivo	Critério de escolha	Resultado final
guardas.py	Aleatório entre as guardas verdadeiras	Sempre [1, 4, 7, 9]
select.go	Escolha entre canais prontos	Contagens somam 1.000, com divisão variável
guardas.hs	Primeira guarda verdadeira	Resultado sempre previsível

Conclusão

Saltos mal posicionados podem ignorar verificações essenciais. As guardas expressam condições de execução, mas a escolha entre elas pode ser ordenada, como em Haskell, ou não determinística, como em Python e em Go.

10 Exercícios de identificação de erros

Objetivo

Prever a saída de cada exercício, explicar o erro presente no código e indicar uma correção.

Análise dos exercícios

Os seis exercícios retomam, em pequena escala, conceitos vistos nos grupos anteriores. A tabela resume o que cada um imprime, a causa do problema e como corrigi-lo; em seguida, cada caso é comentado.

Tabela 5 — Resumo dos exercícios

Arquivo	Saída original	Causa do erro	Saída corrigida
q1.js	"Total: 0012"	for...in entrega índices como texto	60
q2.py	180.0	Desconto zero é falso em d or 10	200.0
Q3.java	"inválido"	switch sem break continua nos casos seguintes	"segunda"

Arquivo	Saída original	Causa do erro	Saída corrigida
q4.c	"saldo insuficiente" e "fim"	Ponto e vírgula após o if	Apenas "fim"
q5.go	"dentro: 20" e "fora: 10"	<code>:=</code> cria variável local	20 nas duas saídas
Q6.java	"0.0%"	Divisão inteira antes da conversão	"70.0%"

q1.js – A saída é "Total: 0012". O `for...in` fornece os índices como strings, e eles são concatenados ao total. A correção é usar `for (const p of precos) total += p`, e então o total passa a ser 60.

q2.py – A saída é 180.0. Em `d or 10`, o desconto zero é considerado falso e acaba substituído por 10%. A correção é testar `if d is None: d = 10`; com desconto zero, o retorno passa a ser 200.0.

Q3.java – A saída é "inválido". O switch começa no caso 2 e prossegue pelos casos seguintes, inclusive o default, sobrescrevendo nome. Corrige-se inserindo `break` após cada caso ou adotando a sintaxe com setas. Para `dia = 2`, a saída correta é "segunda".

q4.c – São impressas duas linhas: "saldo insuficiente" e "fim". O ponto e vírgula logo após o `if` cria um corpo vazio, e o `printf` seguinte executa sempre. A correção é remover esse ponto e vírgula e colocar o `printf` entre chaves. Como 50 não supera 100, apenas "fim" será impresso.

q5.go – As saídas são "dentro: 20" e "fora: 10". A declaração `x := x * 2` cria uma variável local que sombreia a externa. Se a intenção é atualizar a variável original, deve-se usar `x = x * 2`, e assim as duas saídas passam a mostrar 20.

Q6.java – A saída é "0.0%". A divisão entre os inteiros `7 / 10` resulta em zero antes da multiplicação e da atribuição a `double`. A correção é `double taxa = (double) acertos / total * 100`, que gera "70.0%".

Conclusão

Os exercícios mostram que pequenos detalhes de sintaxe, de tipo e de escopo mudam o resultado do programa. Prever a saída e acompanhar as alterações de cada variável é um bom caminho para localizar o problema.

11 Autopesquisa sobre Ruby e Lisp

Objetivo

Explorar outras maneiras de escrever expressões e de percorrer coleções, usando duas linguagens com abordagens bem distintas das estudadas até aqui.

Análise dos arquivos

expressoes.rb – As expressões `1 + 2` e `1.+(2)` produzem 3, o que mostra que o `+` é, em Ruby, uma chamada de método. A potência `2 ** 3 ** 2` associa à direita e resulta em 512. O bloco de `3.times` imprime 0, 1 e 2, e `each_with_index` imprime "0: 10", "1: 20" e "2: 30". Já `select(&:even?)` filtra os números pares do intervalo de 1 a 5 e retorna [2, 4].

expressoes.lisp – A notação prefixa coloca o operador antes dos operandos. $(+ a (* b c))$ gera 23 e $(* (+ a b) c)$ gera 35, e são os parênteses que definem a estrutura da expressão, sem depender de regras de precedência. $(+ 1 2 3 4 5)$ produz 15, pois o operador aceita vários argumentos. O `if` funciona como expressão e retorna 3, porque `a` é positivo.

Conclusão

O Ruby evidencia a ligação entre operadores e métodos, e o Lisp torna explícita a estrutura dos cálculos por meio da notação prefixa e dos parênteses. Os valores 23, 35 e 512 já vistos em outros grupos reaparecem aqui, agora expressos de outra maneira.

Considerações finais

A análise dos 49 programas mostra que a mesma ideia pode ser escrita de várias formas e produzir resultados diferentes. Os principais pontos que atravessam os grupos são:

- **Precedência e associatividade** definem como a expressão é agrupada, e os parênteses são a forma mais segura de deixar essa leitura explícita.
- **Ordem de avaliação e efeitos colaterais** podem não ser garantidos pela linguagem, como em C e Go; separar as instruções evita depender disso.
- **Tipos e conversões** alteram valores por truncamento, estouro, saturação ou coerção, e a igualdade estrita ajuda a evitar surpresas.
- **Curto circuito e valores falsos** economizam cálculos, mas exigem cuidado com o zero, o texto vazio e a ausência de valor.
- **Seleção, repetição e desvios** dependem de detalhes como chaves, `break`, rótulos e a escolha entre guardas, e um erro pequeno pode ignorar verificações importantes.

Em conjunto, os exemplos reforçam que conhecer as regras de cada linguagem é parte essencial de escrever programas corretos e de revisá-los com segurança.